



Republic of the Philippines
PAMANTASAN NG LUNGSOD NG MAYNILA
(University of the City of Manila)
General Luna Street corner Muralla Street
Intramuros, Manila



Bee There!: A Shape Identification Game for Toddlers

An Assembly Project Presented to the Faculty of Computer Science Department

In fulfillment of the requirement for the 2nd Semester

A.Y 2025-2026

Leader:

Capus, Christian Dexthere E.

Members:

Barrieta, Raychel G.

Blanche, Nheil Jewel F.

Bonifacio, James Craven C.

De Vera, Ray Arvin R.

Garcia, John Devonaire L.

Hernandez, John Bennydict R.

Marquez, Antonio Miguel D.

Pambid, Lemony Eve R.

Simon, Lance Aldwin M.

Submitted To:

Prof. Elsa S. Pascual

May 2026

Members' Page

Name	Task/s
Capus, Christian Dexthere E.	- Leader, Programmer
Barrieta, Raychel G.	- UI/UX Designer, Booth Design
Blanche, Nheil Jewel F.	- Documentation, Programmer
Bonifacio, James Craven C.	- Sounds, Programmer
De Vera, Ray	- Documentation, Slides
Garcia, John Devonaire L.	- Documentation, Brochure
Hernandez, John Bennydict R.	- Documentation
Marquez, Antonio Miguel D.	- Documentation, Booth Design
Pambid, Lemony Eve R.	- UI/UX Designer, Booth Design
Simon, Lance Aldwin M.	- UI/UX Designer, Documentation

Introduction

"Bee There!" is a shape identification game for toddlers developed using 8086 Assembly Language, designed for the old 16-bit x86 DOS environment. It emulates a classroom lesson, transforming it into an engaging experience and demonstrating how low-level concepts of computer organization can be used to create a functional game. By employing direct memory access, hardware interrupts, and registers, the game offers a delightful journey in which players practice matching words to various shape patterns.

In the game, players assist a "bee" character in selecting the correct shape layouts to reach its destination. Built directly on machine-level code, this project showcases the possibility of designing a smooth and colorful user interface, along with a playable game, even within the constraints of a limited computing environment without relying on modern high-level game engines.

Core Game Features

The game runs on low-level x86 Assembly routines built directly on DOS interrupts and VGA hardware access.

- **Singleplayer and Multiplayer Mode:** Players can choose to play alone ("Just Me") or play against a friend ("Me + Friend"). Each player's score and mistake count are tracked independently using separate memory-mapped variables, allowing the game to evaluate both performances at the end of the session.
- **Difficulty Selection:** The game consists of three stages: Baby Bee, Young Bee, and Big Bee. As players progress through each stage, the number of shape pattern choices increases, and the layout of the icons is reshaped, requiring players to pay closer attention.
- **VGA Mode 13h Display:** The game initializes a 640×480 pixel, 256-color display by switching the video adapter into Mode 13h via BIOS interrupt and writing pixel data directly to the VGA framebuffer at memory segment A000h. All graphics are rendered through this low-level memory access.

Game Difficulty Mechanics

The game changes its difficulty based on the level chosen by the player. As the level goes up, the game shows harder shape patterns, longer words, and adds more choices on the screen using the keyboard numbers:

- **Baby Bee (Easy Mode):** In each round, the player sees a written shape name as the target indicator. The player must match it to one of the two shape choices displayed on the screen and press either **1** or **2** to make their selection. The shapes on display are visually distinct to minimize confusion.
- **Young Bee (Medium Mode):** Each round presents three shape options, introducing more shapes and with their positions varied to prevent memorization. The player must read the target indicator, identify the correct shape from the three choices, and press **1**, **2**, or **3**. As the game progresses, shape pairs become more similar, requiring closer attention.

- **Big Bee (Hard Mode):** Each round offers four shape options drawn from a larger pool, including polygons and three-dimensional forms. The player must read the target indicator carefully and select the matching shape by pressing **1, 2, 3, or 4**.

Project Objectives

The main aims for this project are the following:

- To make a functioning 16-bit game that runs smoothly, staying inside older DOS-style memory limits, and using flat binary standards.
- To render shape patterns and graphics straight onto the screen by entering exact pixel memory addresses, instead of relying on modern outside libraries.
- To capture keyboard input immediately, so the numeric selection (1, 2, 3, or 4) is registered by the game without noticeable delays.
- To include a competitive 2-player mode (“ME+FRIENDS”) that lets two users play on the same screen and tracks their individual scores correctly.
- To make a real-time leaderboard system that stores and shows high scores directly within the game’s own memory.

Definition of Terms

The following terms are important to understand how the game works:

8086 Assembly Language - The low-level programming language used to develop the game. It directly governs how the game behaves within system memory, providing precise control over hardware-level operations.

Keyboard Input Polling - A method by which the game continuously monitors the keyboard for player input. The system repeatedly checks whether a number key (1, 2, 3, or 4) has been pressed, using the detected input to evaluate the player's answer.

BIOS Interrupt - A mechanism that temporarily suspends the CPU's current operation to execute a specific system-level task, such as switching the display mode or reading keyboard input. BIOS interrupts are integral to the game's core runtime operations.

Bitmap - An uncompressed image file format used to store visual assets within the game, including character graphics such as the bee. Because the files are uncompressed, they require no additional processing to decode, allowing them to load directly and efficiently.

Architecture and Organization

Buffer Memory - A designated area in memory used to temporarily hold data, such as a player's score, before it is rendered on screen. This intermediary storage helps maintain smooth and consistent game performance.

DOSBox - An emulation program that allows the game to run on modern computers. Because the game was built for an older operating environment, DOSBox recreates the necessary system conditions for it to function correctly.

Flat Binary - A file format consisting of raw machine instructions with no additional metadata or structure. The game is distributed as a flat binary, which allows it to be loaded and executed directly by the processor with minimal overhead.

Graphic Sprite - A movable image rendered on screen to represent a game character or object. The game uses sprites, such as the bee character, to simulate motion and bring visual elements to life.

NASM (Netwide Assembler) - A software tool that translates the game's assembly language source code into machine-readable instructions. NASM is a required part of the build process, producing the executable binary that runs the game.

State Machine - A programming design pattern used to manage the game's navigation between distinct screens. Each screen - the main menu, player setup, gameplay, and leaderboard — corresponds to a defined state, and the system transitions between them in a controlled and predictable sequence.

VGA Mode 13h - A standard graphics mode that configures the display to a resolution of 320×200 pixels with support for up to 256 simultaneous colors. The game operates entirely within this mode, which defines its visual output and rendering capabilities.

How To Play

The main goal of the game is to help a little bee travel safely through each stage by matching words with the correct shape patterns. You earn points for every correct answer, which helps you progress and assists the bee in reaching its destination.

Step 1: Game Initialization & Profile Setup

1. Start the game: main.com in your DOSBox to see the main title screen.
2. Begin setup: Press the P key on your keyboard to start setting up your profile.
3. Choose the Game Type:
 - Press 1 for "Me" (Singleplayer)
 - Press 2 for "Me + Friend" (Multiplayer)
4. Enter your name: Type your name on the keyboard and press ENTER to save it.

Step 2: Select Your Difficulty

Select the difficulty level you want for the game:

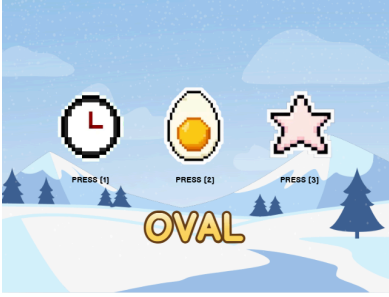
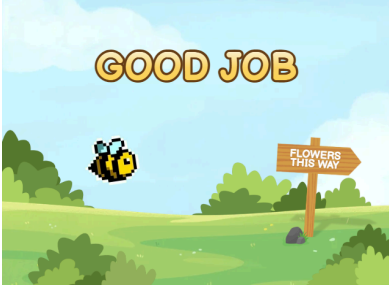
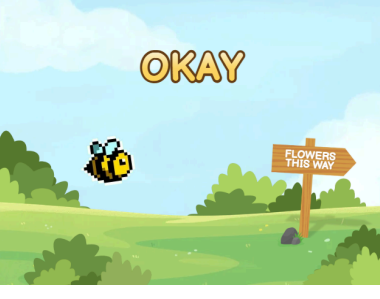
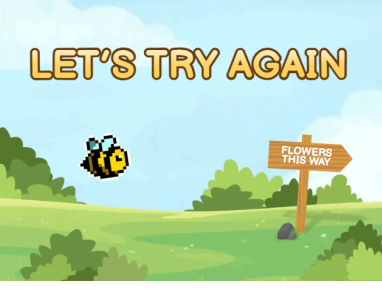
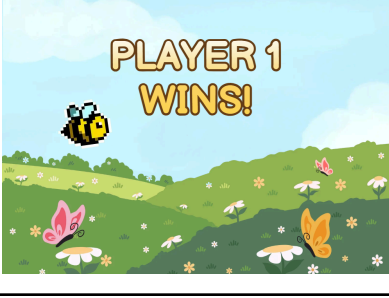
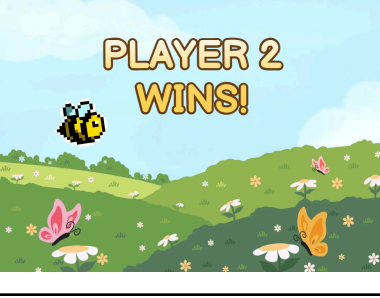
- Press 1 for Baby Bee (Easy): Shapes are basic and instantly recognizable. Players are shown 2 shape choices on the screen (press 1 or 2).
- Press 2 for Young Bee (Medium): Pairs similar-looking shapes. Players are shown 3 shapes to choose from on the screen (press 1, 2, or 3).
- Press 3 for Big Bee (Hard): Introduces more visually distinct and geometric shapes. Players have 4 shapes to choose from on the screen (press 1, 2, 3, or 4).

Step 3: Match the Shape to the Given Word

- Read the shape name: Look at the word at the bottom of the screen to see what shape you need to identify.
- Examine the options: Review the patterns displayed on the screen, which are numbered underneath.
- Submit your answer: Press the corresponding number key on your keyboard to submit your answer.

STORYBOARD / UI DESIGN

Title Screen	Leaderboard	Gamemode Selection
		
Name Input (Singleplayer)	Name Input (Multiplayer)	Difficulty Selection
		
How To Play	Game Introduction	Baby Bee Mode Preview

		
Young Bee Mode Preview	Big Bee Mode Preview	“Perfect” Ending
		
“Good Job” Ending	“Okay” Ending	“Let’s Try Again” Ending
		
“Player 1 Wins” Ending	“Player 2 Wins” Ending	“It’s a Draw” Ending
		

UPDATED CODE IN GITHUB URL

HOW TO INSTALL THE GAME

To execute the 16-bit retro architecture of "BEE THERE!" on modern systems, a specialized emulator must be used to recreate legacy environmental boundaries. This deployment guide provides a direct, step-by-step walkthrough for configuring your software workspace, mounting file structures, and compiling source maps cleanly. Following these simple procedures ensures that hardware loops, display registers, and interface features load smoothly for an optimal gameplay experience.

What You Need Before Installing

Make sure you have downloaded and prepared the following components on your computer:

- The Project Folder: The main directory where all your game code files and image assets are saved.
 - Your File Path: C:\Users\Downloads\Programs\Programs
- DOSBox Emulator: A software that opens a retro MS-DOS terminal environment window.
- NASM Compiler Engine: The tool named nasm.exe that compiles assembly code. Important: Put this nasm.exe file inside your project folder so it works easily.

Step-by-Step Guide to Mount and Open the Folder

Step 1: Open DOSBox

Double-click the DOSBox shortcut icon on your desktop. A black screen command prompt window will open up showing this default drive on the screen:



Step 2: Link Your Project Folder (Mounting)

You need to tell DOSBox where your game files are located on your actual computer. Type this command inside the prompt and press Enter:



```
DOS
```

```
mount d "C:\Users\Charlz\Downloads\Programs\Programs"
```

(If your folder is saved somewhere else, just change the path inside the quotation marks to your actual folder path).

Step 3: Change to Your New Virtual Drive

Right now, you are still inside the default drive Z. To move inside the project folder you just linked, type this command and press Enter:



```
DOS
```

```
d:
```

The prompt line on the left will change from Z:\> to D:\>, which means you are now successfully inside your game directory.

Compiling and Running the Game

Now that your workspace is fully connected, you can build your code and launch the program using these simple steps:

Step A: Compiling the Source Code (Building the Game)

If you made changes to your main code file (MAIN.ASM) and want to apply them, type this compiler command inside Drive D and hit Enter:

```
DOS  
nasm -f bin MAIN.ASM -o MAIN.COM
```

This command tells NASM to strip unnecessary modern headers and pack everything into a small, clean 16-bit machine executable file called MAIN.COM.

Step B: Launching the Game Interface

To load and play your game, type the filename of your executable file and press Enter:

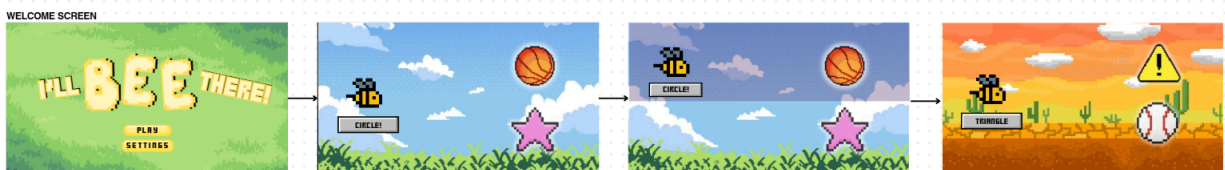
```
DOS  
main.com
```

The DOSBox screen will instantly shift from text mode into colorful graphics mode (VGA Mode 13h with 320 x 200 screen resolution), and you can start playing "BEE THERE!" right away.

PICTURES DURING DEVELOPMENT

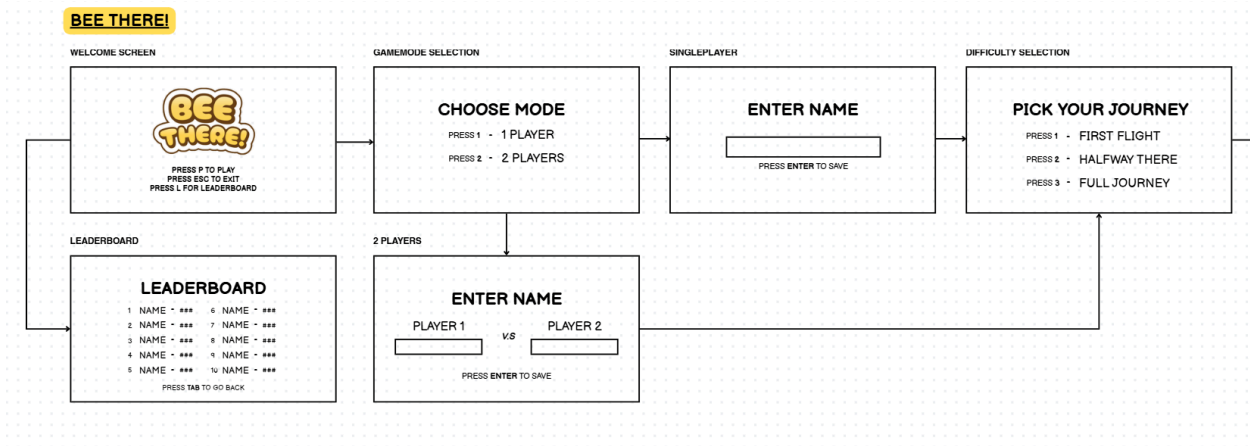
Draft UI and Game Mechanics

The following wireframes present the initial game design and controls for "Bee There!" The project originally began under the title "I'll Bee There!" and features graphics with a heavily pixelated theme, emphasizing the retro aesthetic the team initially intended. The original gameplay mechanics required players to use the up and down arrow keys to move the bee character and align it with the corresponding shape's word.



Finalized Wireframes

Following the first consultation, the team drafted another set of wireframes with the suggestions made in mind. The game added a leaderboard, game mode and difficulty selection, name input, and dedicated soundtracks. These wireframes were then implemented in Canva for the project’s official graphics.



Main Gameplay Mechanics

The attached images illustrate the final gameplay for each difficulty level of the game. The easy mode, Baby Bee, offers two shape choices that are easily recognizable by toddlers. In contrast, Young Bee, the medium difficulty, features three shapes and includes similar-looking shapes together. Finally, the hard difficulty introduces four shapes, incorporating more complex words and geometric shapes into the mix.

